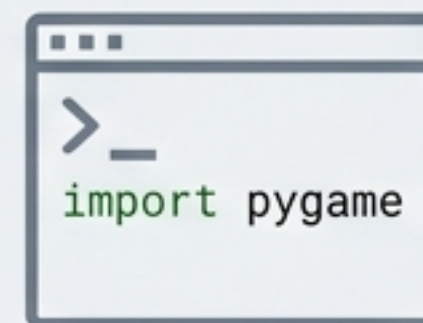


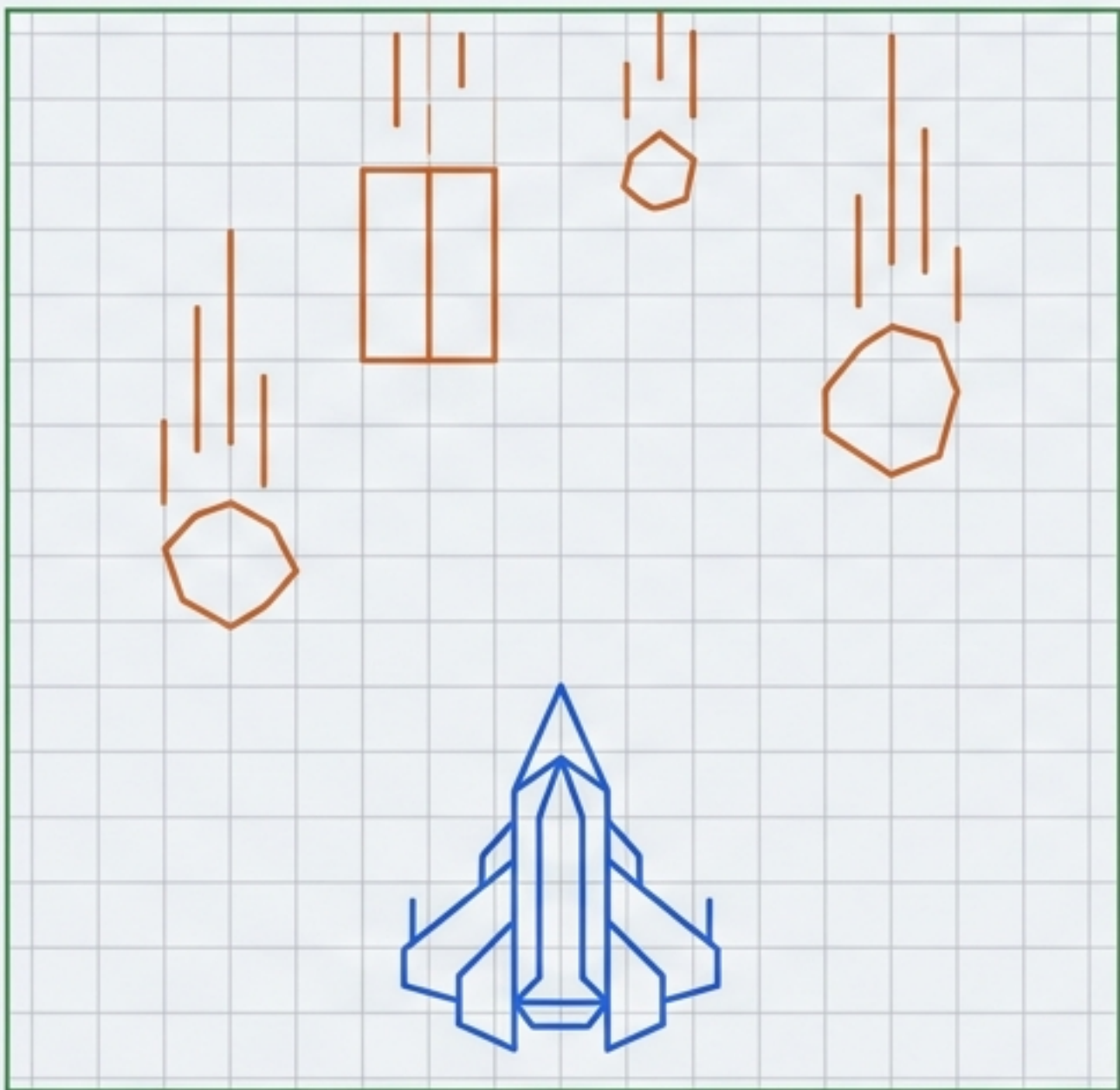
StarshipMeteors: Arquitetura de um Game em Python

Desconstruindo Programação Orientada a Objetos e **Game Loops** com Pygame



Sumário da Execução

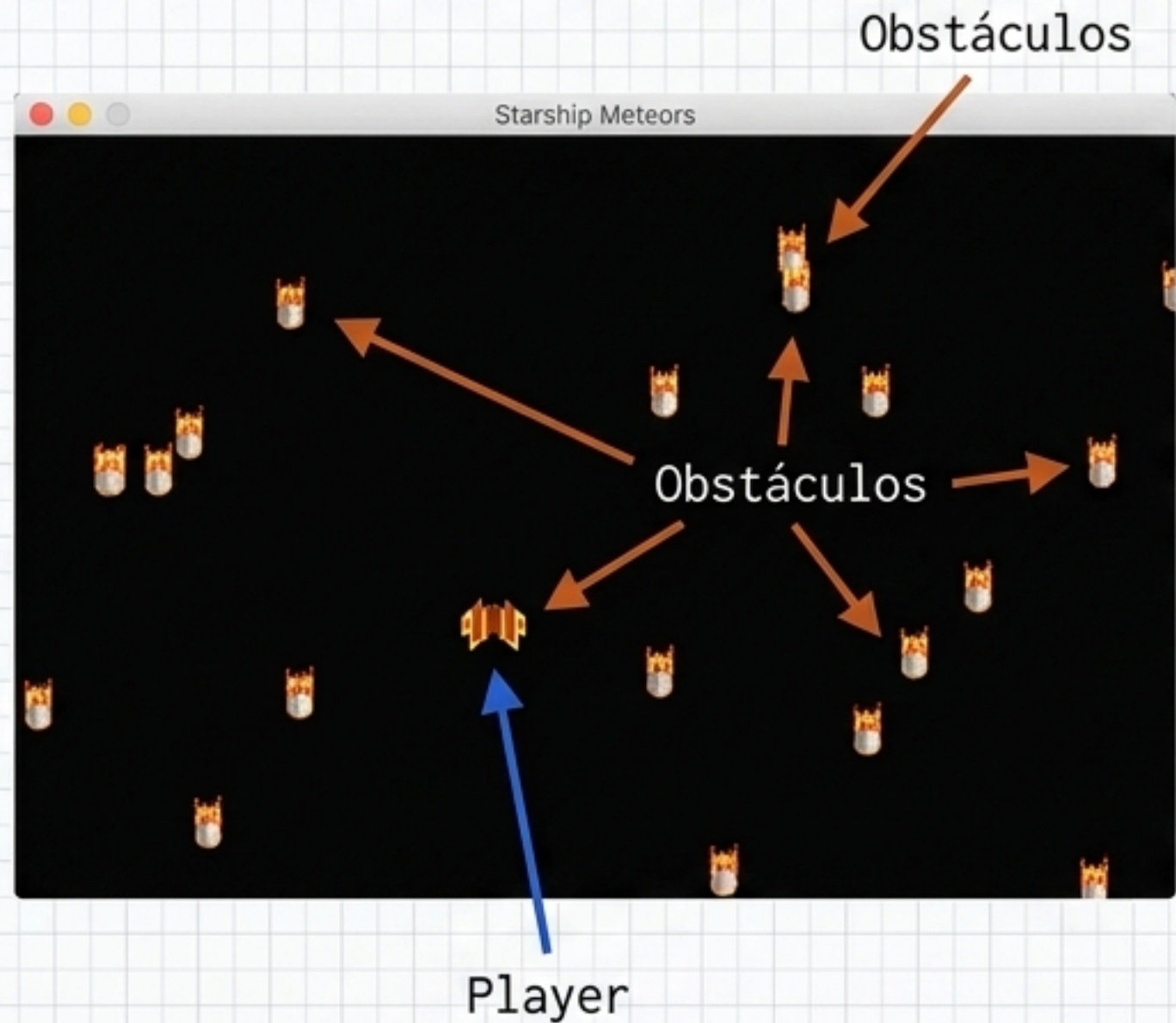
0 Canvas



0 Editor

- **1. Visão Geral & Arquitetura:**
0 Jogo e a Hierarquia de Classes (OOP).
- **2. O Coração do Jogo:**
0 Game Loop e a Renderização.
- **3. Entidades em Ação:**
Matriz Nave vs. Meteoros e Lógica Cartesiana.
- **4. Sistemas Avançados:**
Detecção de Colisão, Spawns e Interrupções.
- **5. Resumo da Compilação:**
Síntese de Aprendizado.

0 Escopo do Projeto: Starship Meteors



Parâmetros do Sistema



Objetivo

Sobreviver por 1.000 ciclos de execução.



Ameaça

Chuva constante de meteoros que aumenta progressivamente.



Mecânica Core

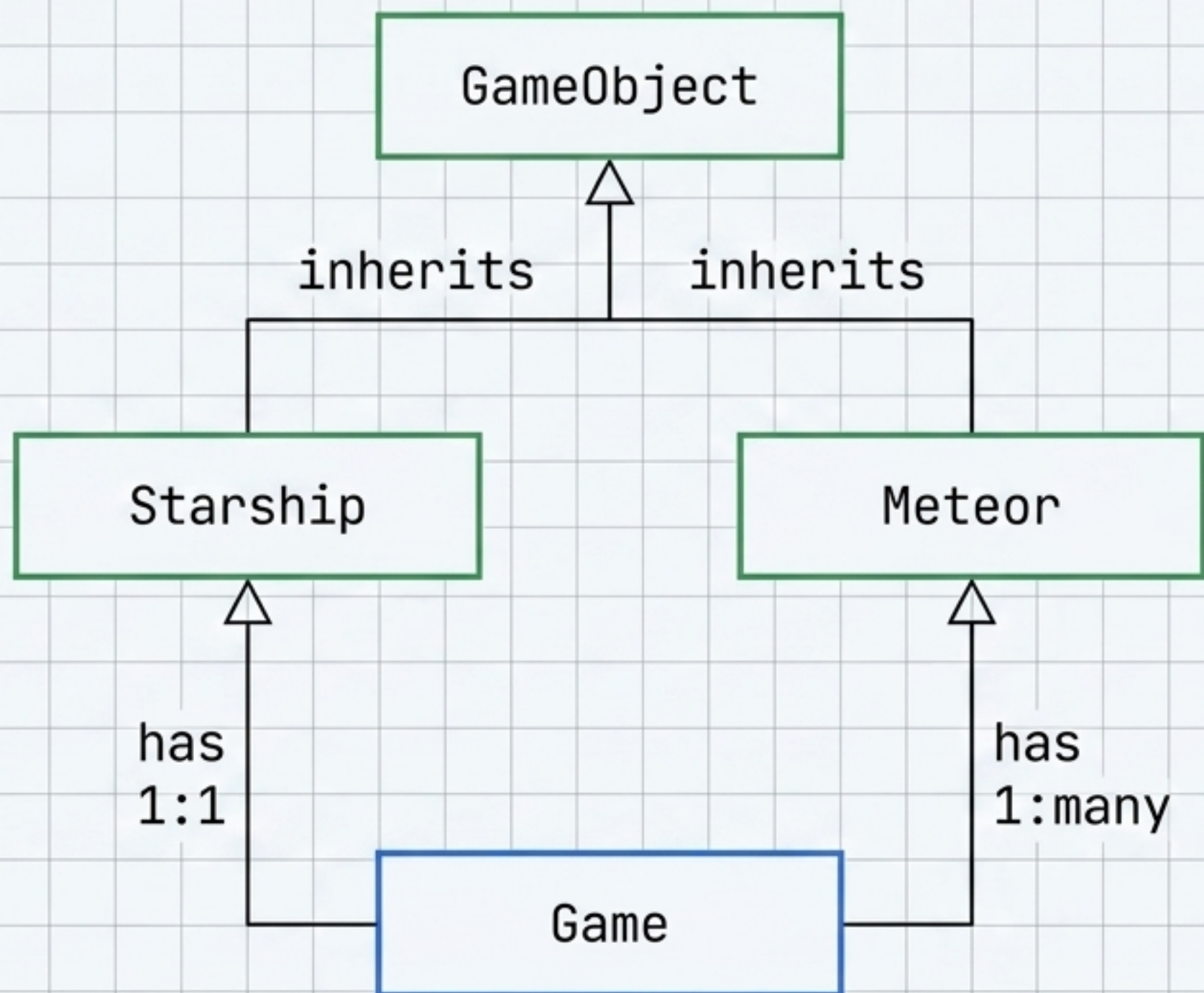
Evitar colisões e gerenciar a movimentação num plano 2D.



Tecnologia

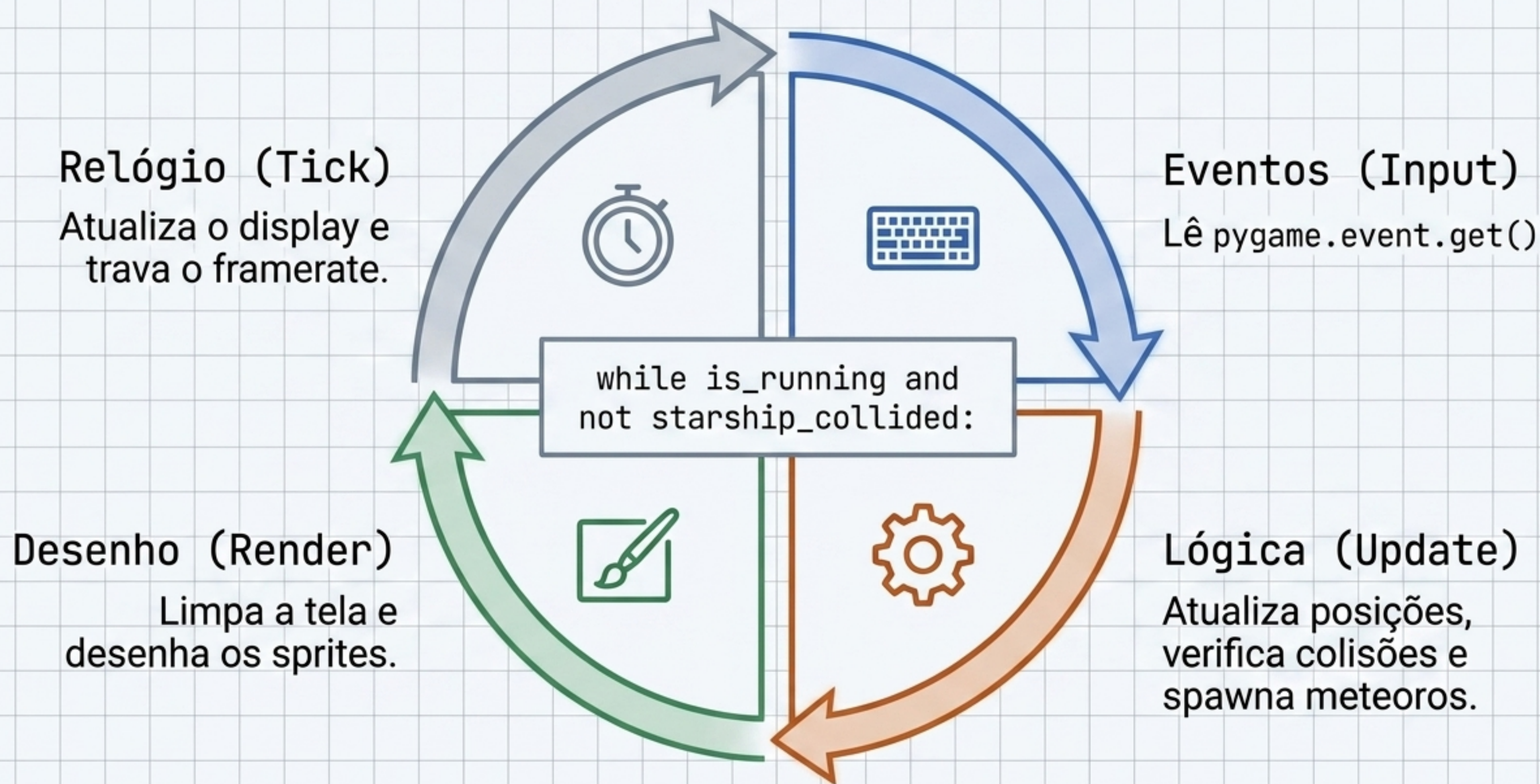
`Python 3` + Biblioteca
`Pygame` (30 FPS).

Fundações P00: A Hierarquia de Classes



Por que usar classes?
Centralização de dados
e facilidade para
instanciar múltiplos
(meteoros) de forma
independente.

A Anatomia do Game Loop Principal



A Classe Base: Traduzindo Imagem em Matemática

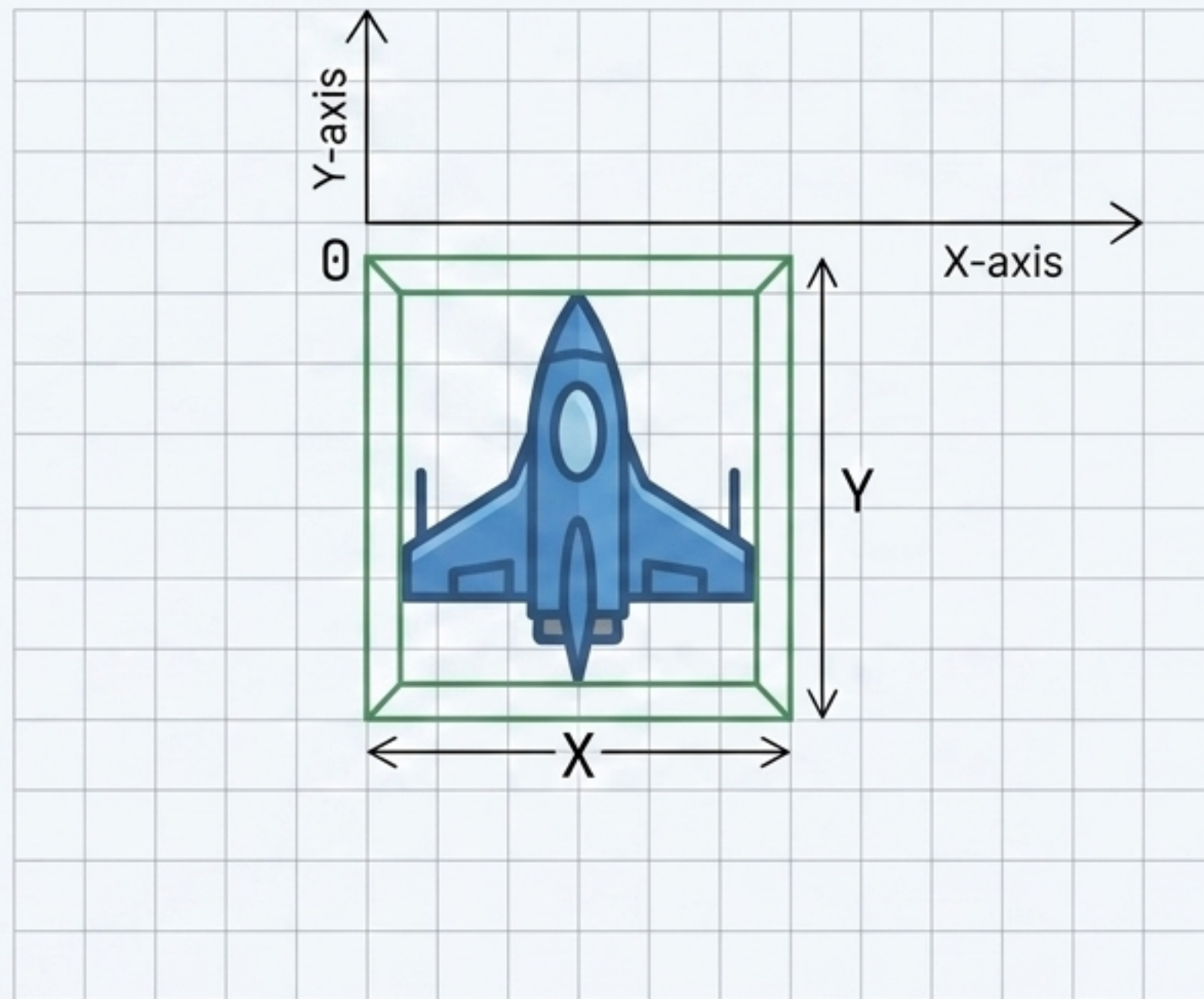
0 Editor

`load_image()` - Carrega o arquivo PNG e extrai dimensões.

`rect()` - Retorna um `pygame.Rect(x, y, width, height)`.

`draw()` - Usa `blit()` para transferir a imagem para o display.

0 Canvas

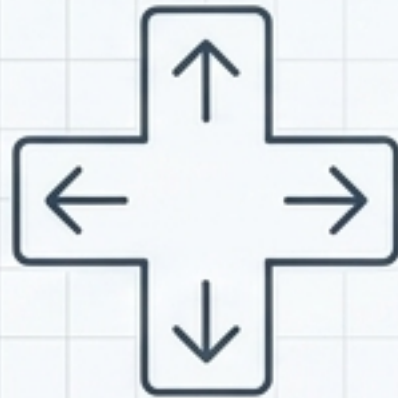


Matriz de Entidades: Nave vs. Meteoros

Característica	Starship	Meteor
Instanciação	Posição fixa inferior central.	Posição Y fixa no topo, X aleatório.
Movimentação	Omnidirecional, velocidade fixa (10).	Unidirecional (para baixo), velocidade aleatória (1 a 5).
Limites da Tela	Colisão rígida (não pode sair da tela).	Wrap-around (retorna ao topo ao passar do limite).
Relação com Game	Atributo único (self.starship).	Lista gerada dinamicamente (self.meteors).

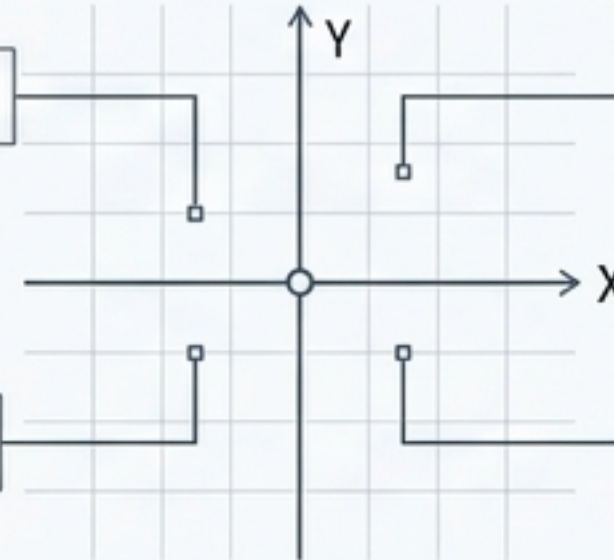
Movimentação: Matemática Cartesiana e Renderização

Lógica Cartesiana



$$\text{ESQUERDA} = x - \text{VELOCIDADE}$$

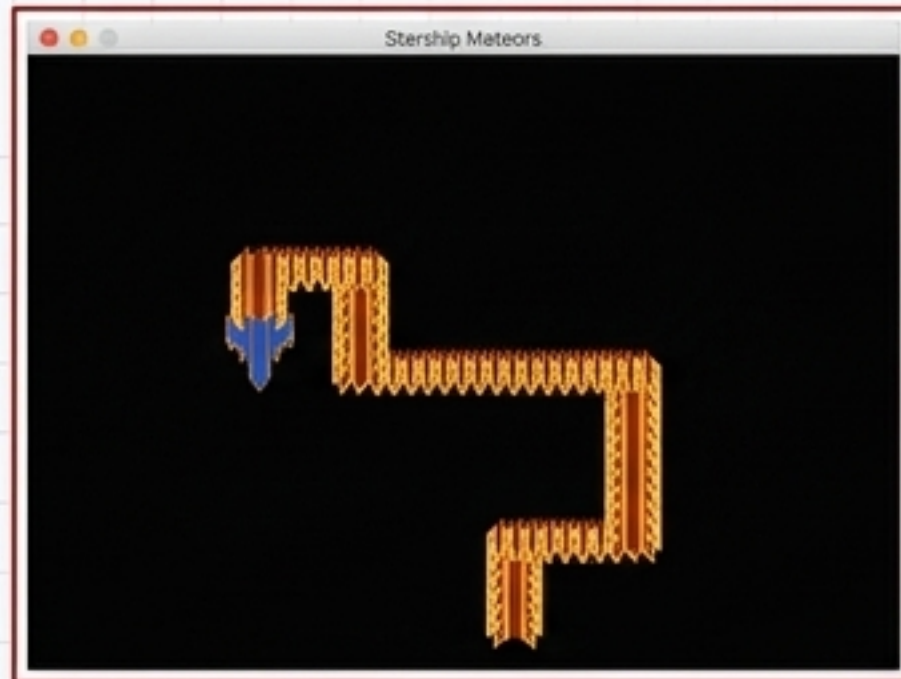
$$\text{DIREITA} = x + \text{VELOCIDADE}$$



$$\text{CIMA} = y - \text{VELOCIDADE}$$

$$\text{BAIXO} = y + \text{VELOCIDADE}$$

0 Paradoxo da Renderização



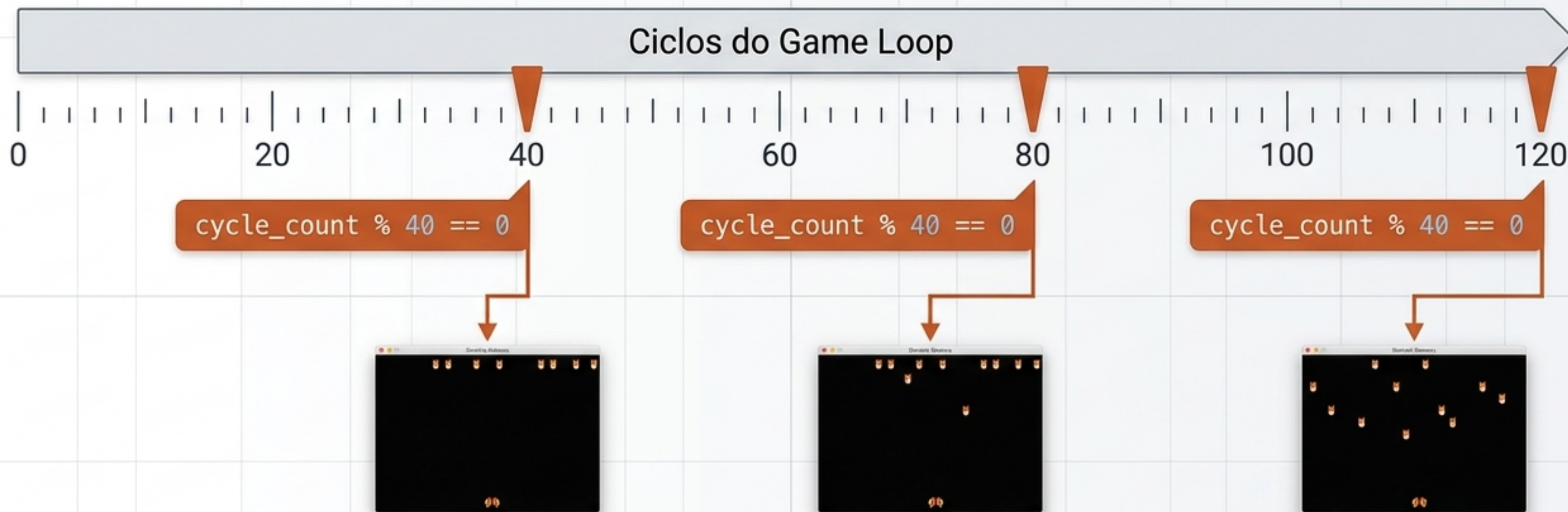
Sem limpar a tela
(Rastro de pixels)

Takeaway: 0 movimento é uma ilusão criada por apagar a tela inteira e redesenhar os sprites em novas posições matemáticas 30 vezes por segundo.



Usando `.fill()` antes
de redesenhar

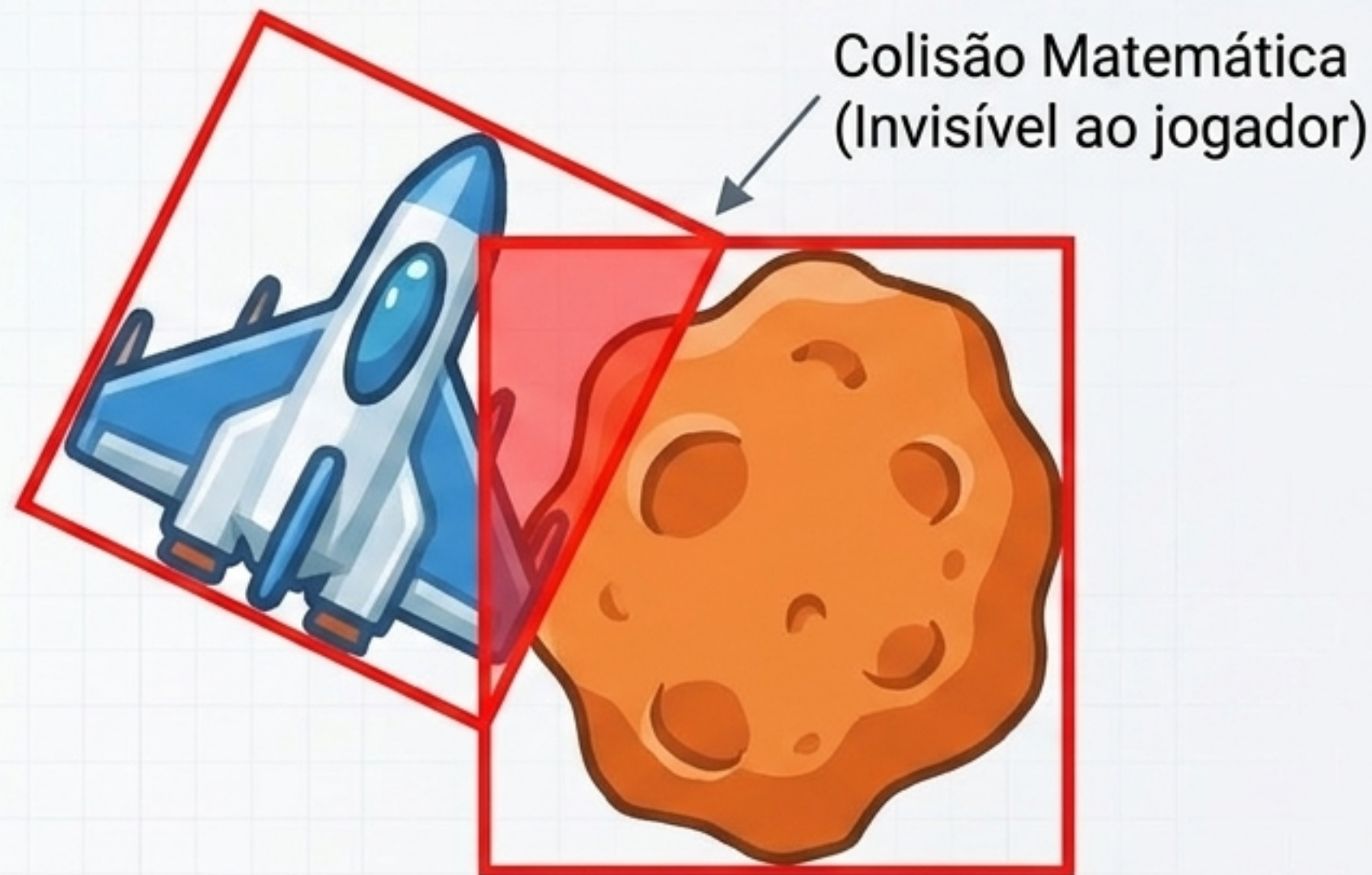
Progressão de Dificuldade: Spawns Temporizados



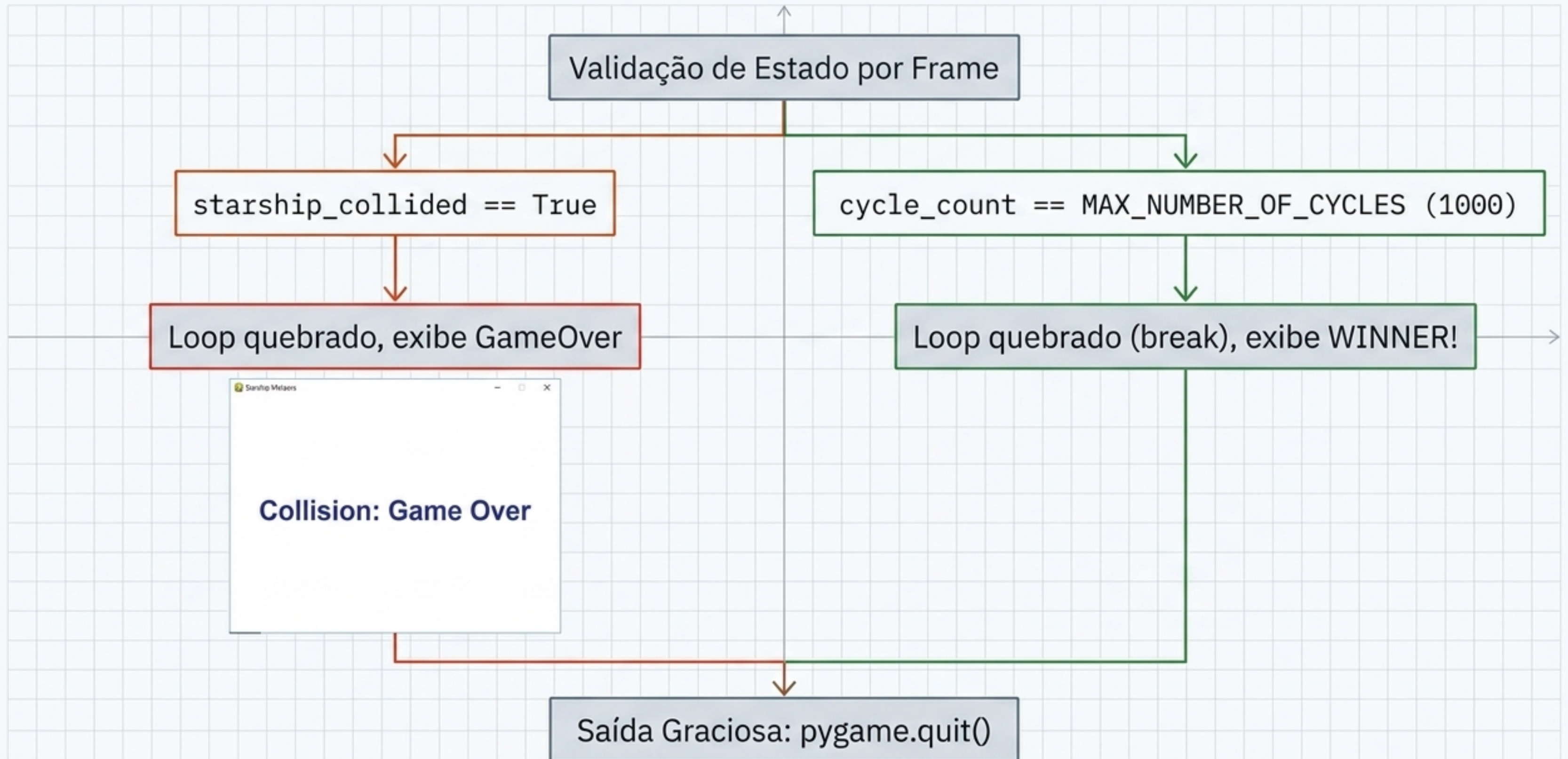
O operador Módulo (%) cria um compasso rítmico perfeito sem timers complexos. A cada 40 frames, a divisão exata engatilha a adição de um novo objeto **Meteor** à lista ativa.

Detecção de Colisões: Interseção de Retângulos

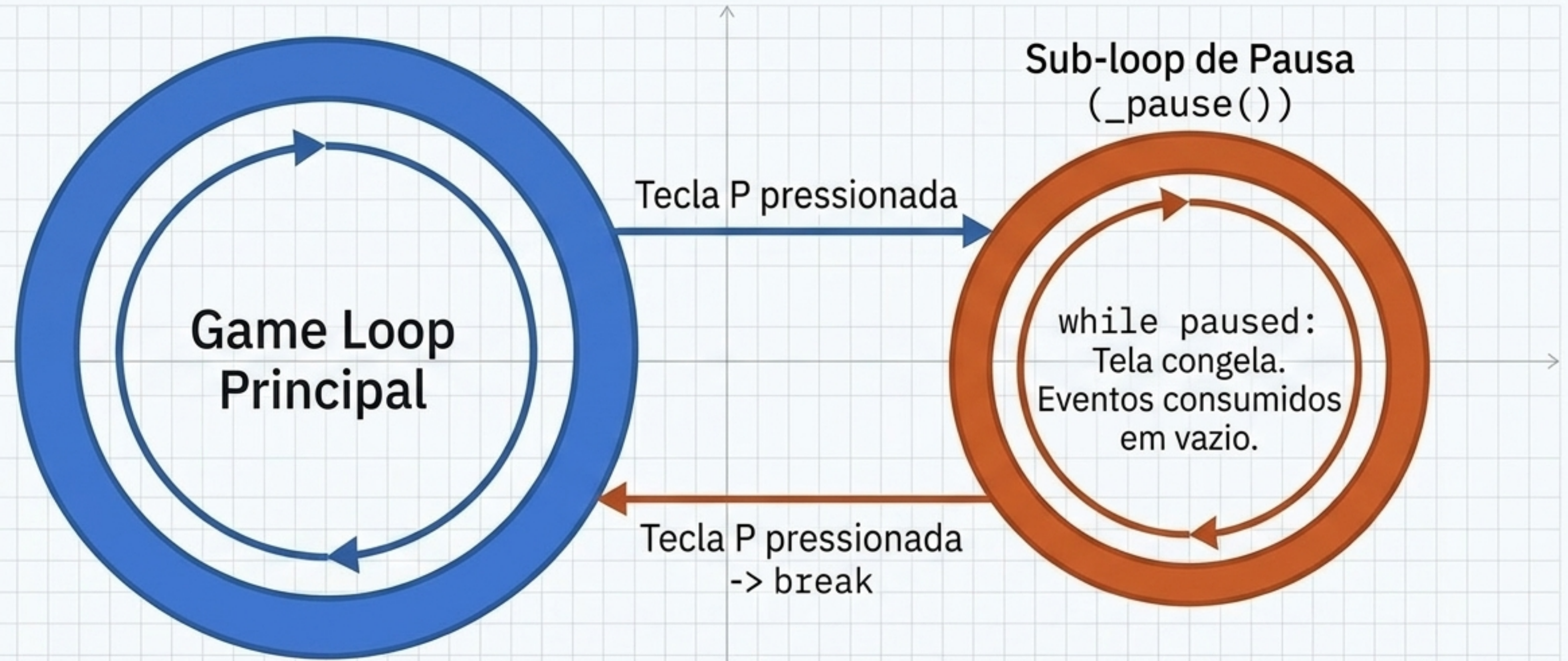
1. Itera sobre a lista de meteoros.
2. Chama `starship.rect().collidirect(meteor.rect())`
3. Se houver sobreposição de pixels no plano (X, Y), retorna True.



Gerenciamento de Estado: Vitória vs. Derrota

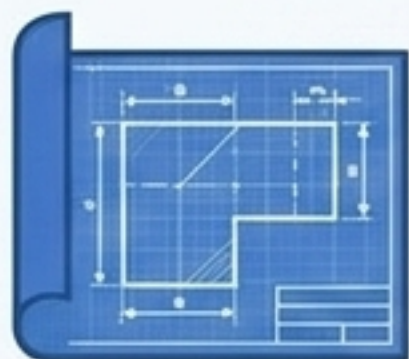


Interrupções de Loop: Implementando a Pausa



Pausar não é "parar o tempo". É aprisionar o programa em um loop secundário infinito que ignora a função de desenho até o botão de liberação ser acionado.

Resumo: A Sinfonia de um Game Engine Simples



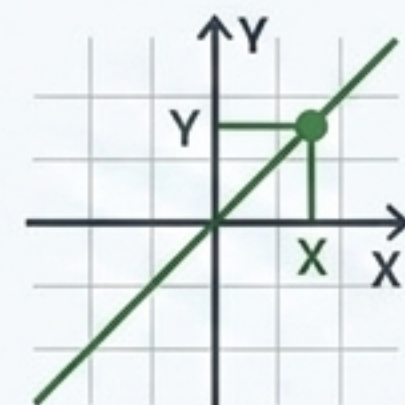
Objetos como Atores

P^{OO} transforma entidades em objetos que conhecem seu formato, aparência e comportamento.



Ilusão de Movimento

O **Loop** é a fundação. Movimento é a destruição da tela seguida da **reconstrução matemática** a 30 *frames* por segundo.



Lógica Interconectada

Jogos 2D são a matemática do **plano cartesiano** gerenciada por **matrizes de colisão** e **relógios modulares**.

A tela é seu Canvas. As classes são seus atores. O loop é o tempo. Construa.